Homelab Infrastructure Documentation: Proxmox Host

Documented July 2026

Intent for this project

I built this homelab to have a real, hands-on environment for cybersecurity and IT administration practice, rather than just studying concepts in isolation. It doubles as a portfolio piece since employers in defensive security and sysadmin roles care a lot about seeing actual infrastructure work, not just certifications or coursework. Finally, I believe it'll be a genuinely good project for learning, honing skills learned throughout University, and experimenting with avenues I enjoy.

This project started July 4, 2026, and is actively evolving. This document gets updated as new pieces get added.

Why Proxmox

I chose Proxmox VE over alternatives like ESXi or running everything in Docker on bare Linux for two main reasons. First, it is free and open source, so there is no licensing cost or restriction getting in the way of experimenting. Second, it gives me real flexibility between full VMs and lightweight LXC containers on the same host, which mirrors the kind of virtualization environment I would expect to work with in an actual IT admin or sysadmin role. Learning Proxmox specifically is also directly relevant experience for that kind of job. Furthermore, prior academic experience with Proxmox allowed for an accelerated deployment timeline.

Why not use dedicated hardware?

The machine hosting all of this is a repurposed office PC rather than something bought specifically for homelabbing. Two reasons factored into that decision. First, it kept the cost of the project at basically zero since the hardware was already sitting unused. Second, it forces me to actually think about resource constraints, since I am working with 16GB of RAM total rather than an unlimited budget of compute. Having to plan around that, deciding what runs full time versus on demand, is its own kind of practical systems administration skill.

Total hardware cost: $0. The machine was already sitting unused, repurposed instead of bought.

Physical Setup

The machine sits next to my main desktop. It runs headless, no monitor, mouse, or keyboard plugged in, since everything is managed through the Proxmox web UI over the local network.

Hardware specifications

CPU	Intel Core i5-6600 @ 3.30GHz, 4 cores, 1 socket
RAM	15.54 GiB total
Storage	1TB external SSD
Swap	8.00 GiB
Network interfaces	enp0s31f6 (onboard NIC), enx5065f34d1da3 (added USB NIC)
Boot mode	EFI
Hypervisor	Proxmox VE, pve-manager 9.2.2
Kernel	Linux 7.0.2-6-pve

At the time of writing, the host is running comfortably under load: 0.76% CPU usage across 4 cores, 10.29% RAM usage (1.60 GiB of 15.54 GiB), 4.57% disk usage on the root filesystem, and 0% swap usage. With this in mind, In the future I'll need to use VMs like Wazuh & other tools on demand and not powered on 24/7.
The host currently has a single physical network interface. Initial testing suggested a second interface might already be present due to how Linux named an altname for the primary NIC, but this was confirmed to be a naming artifact, not separate hardware, verified via lsusb, lspci, and ip a. A USB Ethernet adapter has since been sourced to provide the dedicated second interface pfSense will need

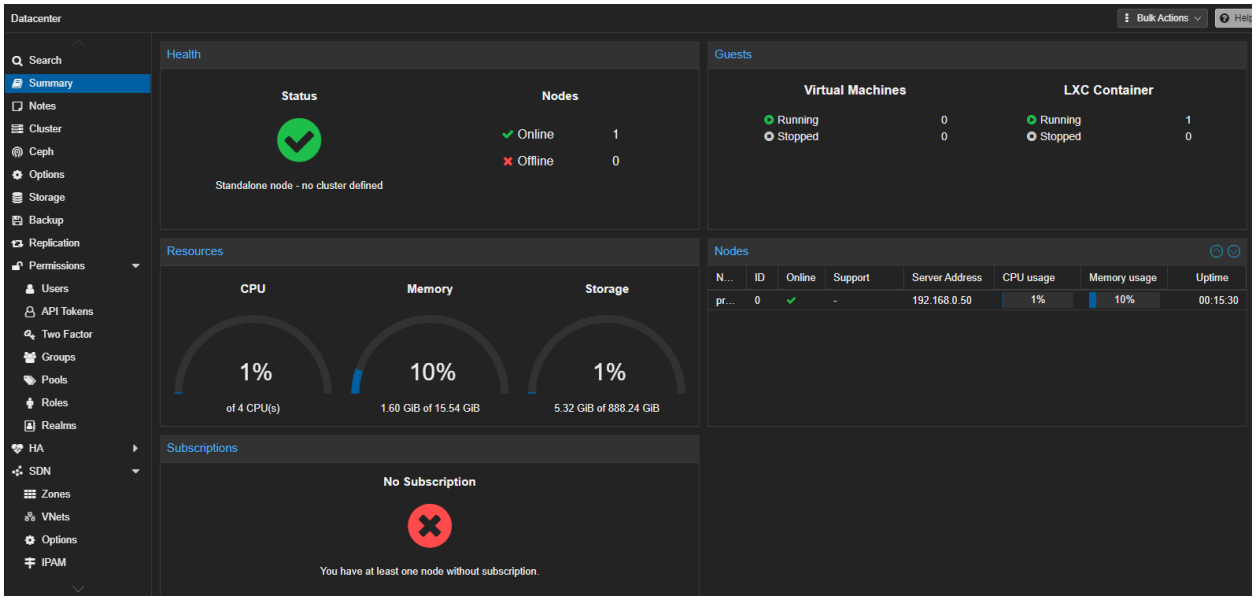
Proxmox environment overview

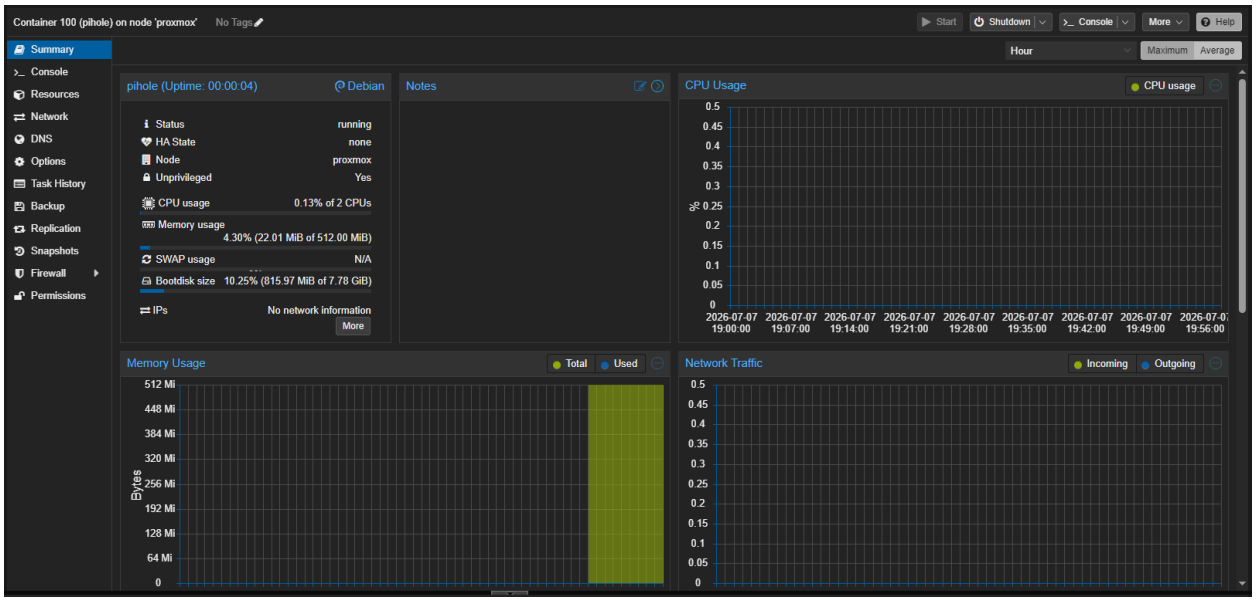
The host is currently running one LXC container, named pihole, on Debian 12. It uses a single storage pool built on the 1TB external SSD for now. As more VMs get added, like Wazuh and the planned Kali box, storage and RAM allocation will need to be tracked more carefully since the 16GB ceiling will be a massive constraint.

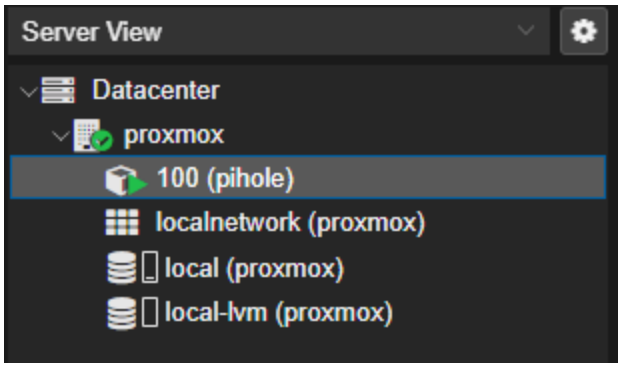
Security Posture

The host itself hasn't been hardened yet. This is next on the list, alongside setting up pfSense. Planned hardening steps include moving the web UI off its default port or restricting it to LAN only, and enabling 2FA on the root account.

Screenshots


Datacenter
Bulk Actions
Search
Summary
Notes
Cluster
Ceph
Options
Storage
Backup
Replication
Permissions
Users
API Tokens
Two Factor
Groups
Pools
Roles
Realms
HA
SDN
Zones
VNets
Options
IPAM
Health
Status
Standalone node - no cluster defined
Nodes
Online 1
Offline 0
Guests
Virtual Machines
Running 0
Stopped 0
LXC Container
Running 1
Stopped 0
Resources
CPU
1%
of 4 CPU(s)
Memory
10%
1.60 GiB of 15.54 GiB
Storage
1%
5.32 GiB of 888.24 GiB
Nodes
N... ID Online Support Server Address CPU usage Memory usage Uptime
pr... 0 - 192.168.0.50 1% 10% 00:15:30
Subscriptions
No Subscription
You have at least one node without subscription.


Container 100 (pihole) on node 'proxmox'
No Tags
Start
Shutdown
Console
More
Help
Summary
Console
Resources
Network
DNS
Options
Task History
Backup
Replication
Snapshots
Firewall
Permissions
Hour
Maximum
Average
pihole (Uptime: 00:00:04)
Debian
Status running
HA State none
Node proxmox
Unprivileged Yes
CPU usage 0.13% of 2 CPUs
Memory usage 4.30% (22.01 MiB of 512.00 MiB)
SWAP usage N/A
Bootdisk size 10.25% (815.97 MiB of 7.78 GiB)
IPs No network information
More
Notes
CPU Usage
CPU usage
%
Memory Usage
Total
Used
Bytes
512 Mi
448 Mi
384 Mi
320 Mi
256 Mi
192 Mi
128 Mi
64 Mi
0
Network Traffic
Incoming
Outgoing


Server View
Datacenter
proxmox
100 (pihole)
localnetwork (proxmox)
local (proxmox)
local-lvm (proxmox)

Planned next steps

- Harden the Proxmox host itself
- pfSense VM
- Wazuh SIEM
- Kali Linux VM
- Jellyfin with GTX 960

Remote collaboration access

As collaborators join the project, network-level access needs to be granted without exposing the home network broadly or handing out direct credentials to internal infrastructure.

The approach used is a Tailscale subnet router, deployed as a dedicated lightweight LXC container rather than installed directly on the Proxmox host. This keeps the hypervisor itself untouched by the remote access layer and isolates that role to a single-purpose container.

Rather than granting full LAN access, collaborator devices are tagged and restricted through Tailscale's access control policy to only the specific services they need, scoped by IP and port rather than by subnet. Everything else on the network remains unreachable to a tagged device, regardless of what the underlying subnet route technically covers, the ACL is the actual enforcement boundary, not the route itself.

This is intentionally a temporary, software-level boundary. Once host hardening and a dedicated pfSense/OPNsense router are in place, this will be replaced with real network segmentation (VLANs), moving the boundary from an access-control policy to physical network architecture.

Proxmox-level permissions (scoped users, resource pools) are the next layer planned, so collaborator access to specific VMs/containers can be similarly restricted at the hypervisor level, not just the network level.